Reviewer Pack — Minimal Rank of Alexander Modules over Circuit Satisfiabilit…

Entry 59e64be32210 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 07:07:18 UTC

Minimal Rank of Alexander Modules over Circuit Satisfiability

Entry ID: `59e64be32210` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 07:07:18 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Alexander Modules) **Field B** (complexity object): Complexity Theory: Boolean Circuit Satisfiability

Statement:

> ['For every Boolean circuit C with n inputs, the minimal rank of its associated Alexander module A(C) is upper-bounded by a function f(n) = O(log n).', 'A falsifier-friendly formulation: If there exists a Boolean circuit C with n ≥ 10 such that the minimal rank of its associated Alexander module A(C) exceeds a constant multiple of log n, then the conjecture is false.', 'The constructive mapping: Convert each input to an oriented simple closed curve in the plane, which represents a vertex in the binary tree of variables for the circuit. The adjacency relation between vertices corresponds to the AND-gates in the circuit. Compute the Alexander module A(C) by applying the Alexander-Grassmannian construction on this configuration.']

Rationale (proposer's reasoning):

> ['Alexander modules are topological invariants that capture the structure of links and knots, which could potentially expose non-trivial relationships between the topology of Boolean circuits and their satisfiability.', 'The minimal rank of an Alexander module is a computable quantity, as it can be calculated from the binary tree representation of the circuit. This makes it a suitable invariant for complexity-theoretic analysis.', 'Alexander modules have not been previously applied to complexity theory, suggesting that this could be a novel approach to understanding computational problems.']

Taxonomy category: `Alexander_Module` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `45224092651a43fb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds and for all n from 1 to 40, the mean minimal rank of the Alexander module A(C) for each circuit C satisfies |mean_min_rank - log(n)| ≤ 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'Alexander modules' AND 'Boolean circuit satisfiability' AND 'minimal rank'` - `'Circuit Satisfiability' AND 'Alexander module' AND 'upper bound on rank'` - `'oriented simple closed curve' AND 'AND-gates' AND 'Alexander-Grassmannian construction'`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return '0'
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return f'({left},{right})'

    def alexander_module(circuit):
        if circuit == '0':
            return [1]
        elif circuit == '1':
            return [-1, 1]
        else:
            left, right = circuit.split(', ')
            A_left = alexander_module(left)
            A_right = alexander_module(right)
            n = len(A_left) + len(A_right) - 2
            A = [0] * (n + 1)
```

```
            for i in range(len(A_left)):
                for j in range(len(A_right)):
                    A[i + j] += A_left[i] * A_right[j]
            return A

    n = random.randint(5, 40)
    circuit = generate_circuit(n)
    A = alexander_module(circuit)
    min_rank = max(abs(x) for x in A if x != 0)

    f_n = math.log2(n + 1)
    metric_value = abs(min_rank - f_n)

    return {
        "metric_name": "minimal_rank",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": metric_value <= 3,
        "counterexample": "" if metric_value <= 3 else f"n={n}, min_rank={min_rank}, f(n)={f_n}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 160, 5))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support_
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"n={first_failing_seed}\" first_failing_seed={fi
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction:.2f}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_acc3c8ca.py", line 66, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_acc3c8ca.py", line 47, in run_trial
    A = alexander_module(circuit)
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_acc3c8ca.py", line 35, in alexander_mod
    left, right = circuit.split(', ')
    ^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the pre-registered support condition. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13212
2	preregistration	ollama_remote	glm4:latest	0	0	5712
3	novelty	ollama_remote	glm4:latest	0	0	4831
4	novelty	ollama_remote	glm4:latest	0	0	5458
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14855
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10044
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11291
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7944
9	judge	ollama_remote	glm4:latest	0	0	12851

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 86198 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/59e64be32210.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/59e64be32210.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/59e64be32210.tar.gz` (if generated)